

Expressive GNNs via super line graph

Giuliano Crucioli

University of Padua

Advisor: Nicolò Navarin

July 15, 2026

Abstract

Graph neural networks (GNNs) are designed to learn functions on graphs. The most famous architectures are based on message passing, but a very strong theoretical limit has been shown: they are not able to distinguish two graphs that are not distinguishable by the Weisfeiler-Lehman test (Morris et al. [5]). I propose and explore a new class of networks with high expressive power inspired by line graph theory that do not use message passing, but transformations called line graph and super line graph. Both allow for the creation of a new graph from a given one. The second transformation amplifies the topological structure of the graph, this means that given two graphs that the Weisfeiler-Lehman test does not distinguish, they become easily distinguishable by some metrics (e.g. number of edges) after the super line graph transformation. I explore the size of the graph after the transformation, the computational cost, the theoretical expressivity, and its applications to construct a new architecture. The goal is to find a new approach to construct an architecture with a firm theoretical foundation. By introducing a different bias, I expect to achieve different performances on tasks where other models fall short.

1 Introduction

Many papers have partially explored the use of the line graph transformation in graph neural networks. The theoretical aspects are collected in the book Line graph and line digraph [2] with all the main properties in graph theory. Moreover, theoretical insights on graph learning have been widely studied [9]. The line graph was used to build new architectures ([10],[6],[3]), especially in link prediction, because thanks to the line graph transformation we can transform a problem on edges into a problem on nodes ([10], [7]). The theoretical properties of the super line graph transformation useful for graph learning, and its application to create a new architecture, have not yet been explored. First, I present the preliminary definitions.

1.1 Preliminary definitions

Let $G = (V, E, X)$ be a graph, where $E \subseteq V \times V$ is the set of edges, V is the set of nodes, $V = \{v_1, v_2, \dots, v_n\}$, $|V| = n$, $|E| = m$, A is the adjacency matrix of the graph, $A \in \mathbb{R}^{n \times n}$ where $A_{ij} = 1$ if $(v_i, v_j) \in E$ and $A_{ij} = 0$ otherwise. $X \in \mathbb{R}^{n \times d}$ is the matrix of the features of the nodes, the i -th row is the feature vector of node i , called $x_{v_i} \in \mathbb{R}^d$, and $N(v) = \{u \in V \text{ s.t. } (u, v) \in E\}$ is the set of neighbors of node v . The Big-O notation $O(\cdot)$ denotes an upper bound for the worst-case growth rate of a function.

1.2 Weisfeiler-Lehman test

The Weisfeiler-Lehman test (or WL), as detailed by Huang et al. [4], is an algorithm that checks if two graphs are isomorphic. It takes as input a graph $G = (V, E, X)$ and outputs a multiset of labels, one for each node. If G_1 and G_2 are isomorphic then $WL(G_1) = WL(G_2)$. The WL succeeds at distinguishing a pair of non-isomorphic graphs G_1, G_2 if $WL(G_1) \neq WL(G_2)$.

We can also define the k - WL that is able to detect more complex structures, by considering k -tuples of nodes of the graphs. It is proven that 2- WL has the same expressive power as 1- WL , but for $k > 2$ we have that the k - WL test is strictly stronger than $(k - 1)$ - WL (and always stronger than 1- WL).

1.3 Line graph and super line graph

As defined in the book Line graph and line digraph [2], let $G = (V, E)$ be an undirected graph.

The line graph of G is defined as a graph that takes the edges of G as nodes, and two nodes are adjacent if the corresponding edges share a vertex in G .

Formally, $L(G) = (V_L, E_L)$ is such that $V_L = E$ and $E_L = \{(e_i, e_j) \mid e_i, e_j \in E, e_i \text{ and } e_j \text{ share a vertex in } G\}$. The super line graph of index k of G is a graph $L_k(G)$ where the nodes are the k -subsets of edges of G and two nodes are adjacent if one edge of the first node is incident with one edge of the second node. Formally for $k = 2$, $L_2(G) = (V_{L_2}, E_{L_2})$ is such that $V_{L_2} = \{\{e_i, e_j\} \mid e_i, e_j \in E, e_i \neq e_j\}$. Every node in $L_2(G)$ represents a distinct pair of edges from G . Two nodes $A, B \in V_{L_2}$ are adjacent if an edge in A is incident with an edge in B (sharing a vertex in G). Note: An edge is not incident with itself.

2 Properties of Super Line Graph

I want to understand how super line graph transformation can impact expressivity and what is the complexity of such transformation, and the memory usage compared with other approach.

2.1 Empirical results

I tested the expressivity of $L_2(\cdot)$ on the *BREC* dataset [8]. It is composed of 400 pairs of graphs. Each pair G_1, G_2 consists of non-isomorphic graphs that cannot be distinguished by the Weisfeiler-Lehman test and, in some cases, by 3-*WL*. There are five types of non-isomorphic graphs: basics, regular, strongly regular, extensions, CFI [8]. I noticed that in this dataset, computing the *WL* test on the super line graph transformation of each pair is strong enough to distinguish many of the non-isomorphic pairs that the standard *WL* test could not. It is always as expressive as 3-*WL*, and sometimes as 4-*WL*. This means that in this dataset for every two graphs G_1, G_2 that 3-*WL* can distinguish, $WL(L_2(G_1)) \neq WL(L_2(G_2))$. Sometimes 3-*WL*(G_1) = 3-*WL*(G_2) but $WL(L_2(G_1)) \neq WL(L_2(G_2))$, but the $WL(L_2(\cdot))$ is never strictly more expressive than 4-*WL*. It has to be considered that in my implementation some super line graph transformations go out of memory so I am not able (for now) to test the whole dataset. None of the 400 pairs is distinguishable by 1-*WL*. Only 51% of the total pairs have been computed. Sometimes given a pair G_1, G_2 not distinguishable by the Weisfeiler-Lehman test, it can be distinguishable by just looking at the corresponding number of edges of the pair $L_2(G_1), L_2(G_2)$. This happens in this dataset 44.4% of the time. Furthermore, out of the 107 pairs that the 3-*WL* test is able to distinguish, 96 of them can be distinguished simply by comparing the number of edges in their corresponding L_2 graphs.

2.2 Dimension of the graph $L_2(G)$

Because the $L_2(G)$ graph risks becoming too large, I analyze its size, especially regarding the number of edges $|E_{L_2}|$, which can vary significantly depending on the structure of G . It is known that $\text{diam}(L_k(G)) \leq 2$. Given $G = (V, E)$ with n nodes and m edges, $|V_{L_2(G)}| = \binom{m}{2}$. In the worst case (Ch. 15 of [2]), for the complete graph, $|E_{L_2(K_n)}| = \Theta(n^7)$, and for the cycle graph $|E_{L_2(C_n)}| = \Theta(n^3)$. I aim to write $|E_{L_2}|$ as a function of the mean degree of the nodes in G , where given a graph G , the degree of a node is the number of edges incident to it. I obtain this result.

Lemma 2.1. *Given $G = (V, E)$, $m = |E|$, $|V| = n$, $d(v)$ is the degree of $v \in V$, we can define $M = \frac{1}{n} \sum_{v \in V} d(v)$, and $\sigma^2 = \frac{1}{n} (\sum_{v \in V} d(v)^2) - M^2$. Then*

$$|E_{L_2}| = O\left(m^3 M + \frac{m^3 \sigma^2}{M}\right) = O(n^3 M^4 + n^3 M^2 \sigma^2).$$

Proof in Appendix A.

Remark 2.2. *We can notice that for k -regular graphs (with small k), Lemma 2.1 guarantees obtaining $|E_{L_2}| = O(n^3)$.*

I compute $\mathbb{E}[|E_{L_2(G(n,p))}|]$ where $G(n, p)$ is a graph generated by an Erdős-Rényi model, allowing me to study an entire class of graphs that are representative of real-world datasets.

Lemma 2.3. *Given $G = G(n, p)$ a random graph, where for all pairs of nodes $u, v \in V$, p is the probability that they are adjacent and $n = |V|$. Then*

$$\mathbb{E}[|E_{L_2(G)}|] = O(n^7 p^4).$$

Proof in Appendix A.

Notice that if $p \approx \frac{1}{n}$ then $\mathbb{E}[|E_{L_2(G)}|] = O(n^3)$. Remember that the critical point to get a random graph not connected with high probability is $p \ll \log(n)/n$. Now we define Barabási-Albert model (BA model).

Definition 2.4. (BA model) Given a starting graph $G_0 = (V_0, E_0)$, we can set two terms: $\bar{m}$ and T , both positive integers and s.t. $\bar{m} \leq |V_0|$. Starting from G_t each step G_{t+1} is built by adding a new node and connecting it with $\bar{m}$ other nodes. This connection is distributed as an extraction without replacement with this distribution $(\frac{d(v_1)}{\sum d(v_i)}, \dots, \frac{d(v_{n_0+t})}{\sum d(v_i)})$. G_T is the graph created by the BA model.

By using a similar reasoning as before I tried to give a bound to the dimension of $\mathbb{E}[|E_{L_2(G)}|]$, that represent the class of random graphs with hubs. I found one but only in the case of many steps and $\bar{m} \gg 1$. Because, for example, in case the starting graph is $G_0 = K_{1, n_0-1}$, $\bar{m} = 1$ and $n_0 \gg 0$, the final graph will be very similar to $K_{1, T+n_0-1}$. With a similar configuration we have with high probability this degenerative case, which gets us a star graph as a result. We can observe that in case of a start graph $K_{1, n}$, then $|V(L_2(G))| = O(n^2)$, $|E(L_2(G))| = O(n^4)$, because $L_2(K_{1, n}) = K_{\binom{n}{2}}$.

Observation 2.5. Given a graph G_T generated by the AB model, with $T \gg 0$ and $\bar{m} \gg 1$, then:

$$\mathbb{E}[|E_{L_2(G)}|] = O(n^3 \bar{m}^4 \log(n)).$$

Proof sketch in Appendix A.

The sketch of the proof uses many approximations, so I verify the result by doing some empirical tests that are summarized in Appendix B that helps us also visualize the poor upperbound for the degenerative case.

2.3 Expressive results

Given the empirical result that the number of edges after the transformation is so expressive I tried to find some expressive result.

Lemma 2.6. Let G_1 and G_2 be non-isomorphic, k -regular, triangle-free graphs on n vertices and m edges that contain a different number of induced C_4 subgraphs. Then the number of nodes with degree $2k - 6$ in $L_2(G_1)$ and $L_2(G_2)$ is different.

Sketch of the proof in Appendix A. Following the same logic as the proof of Lemma 2.6, counting the degree of the nodes after the transformation suggests the possibility of identifying a certain structure that $WL(L_2(G))$ can distinguish and $3-WL$ cannot. We can also loose the hypothesis of a "triangle-free" with the right adjustment.

3 summary of the results

This is the summary of the number of edges given different configurations: Given this summary we can

Graph Type	Parameters	Bound for $ E_{L_2} $
General Graph G	n, m, M, σ^2	$O(n^3 M^4 + n^3 M^2 \sigma^2)$
Erdős-Rényi $G(n, p)$	n, p	$\mathbb{E}[E_{L_2}] = O(n^7 p^4)$
BA Model G_T	$n, \bar{m} \gg 0, T \gg 0$	$\mathbb{E}[E_{L_2}] = O(n^3 \bar{m}^4 \log(n))$

consider that constructing $L_2(G)$ and doing the Weisfeiler-Lehman test costs $O(V(L_2(G)) + E(L_2(G)))$ which hopefully is computationally faster than $3 - WL$.

4 Computational results

I present an algorithm capable of computing $L_2(G)$ in $O(|V_{L_2}| + |E_{L_2}|)$, which is the best possible bound. I implemented it in C++ with a Python wrapper. The algorithm description is detailed in the appendix Appendix A.

5 Models based on $L_2(G)$

I define an architecture that leverages the super line graph transformation. The super line graph of index 2 is computed during the preprocessing phase to avoid redundant calculations. While various architectures have been explored, only one is detailed here.

Given $G = (V, E, X, X_E)$, where $X \in \mathbb{R}^{n \times d_v}$ represents the node features and $X_E \in \mathbb{R}^{m \times d_e}$ represents the edge features, the architecture first computes initial hidden representations of the nodes and edges. This is achieved by applying a linear transformation, a random projection, or an embedding for categorical values. These results are stored in $H_V^{(0)} \in \mathbb{R}^{n \times d}$ and $H_E^{(0)} \in \mathbb{R}^{m \times d}$. Then, the model computes the hidden representation of the nodes in the super line graph. Since $h_A = h_{\{e,f\}} = h_{\{\{u1,u2\},\{u3,u4\}\}}$, a method that is invariant to edge and vertex isomorphism is required. One such method is as follows:

The representation $h_e^{(1)} = \text{SetTransformer}_1(h_{u1}^{(0)}, h_{u2}^{(0)}) + h_e^{(0)}$ is computed to encode edge features. Then, h_A is obtained via $\text{SetTransformer}_2(h_e^{(1)}, h_f^{(1)}) + \frac{1}{4} \sum_{i=1,2,3,4} h_{ui}^{(0)}$, where the summation term acts as a skip connection to preserve information from previous layers and improve gradient flow. The same dimension is used for every hidden representation.

Thus, $H^{(1)} \in \mathbb{R}^{|V_{L_2}| \times d}$ is obtained, and $H^{(2)}$ is built using methods that encode the structure of $L_2(G)$, such as a GCN or SGC.

A readout function is defined by employing global mean pooling, global attention pooling, or a set transformer on the last hidden representation.

Finally, a multilayer perceptron is used to obtain the result $Y = \text{MLP}(H_G)$.

6 Experiments

I implemented the project in Python, PyTorch Geometric. I employ a 60/20/20 split for training, validation, and testing. The code is available on my GitHub (<https://github.com/Giulianocrucio/Thesis-Presentation>). This snapshot represents ongoing work, with statistical significance testing and full validation still in progress.

Table 1: Results are reported in Accuracy (Acc.) for Enzymes, Mutag, and NCI1 (higher is better), and Mean Absolute Error (MAE) for Zinc (lower is better).

Dataset	GCN	mod_slg2_v2	slg_advanced
Enzymes	0.4667	0.4833	0.3167
Mutag	0.8000	0.8000	0.9000
NCI1	0.7932	0.7470	0.7543
Zinc	0.4273	0.4232	0.5280

The architecture slg_advanced is the one described in Section 5, while mod_slg2_v2 is a variant that mixes GCN with the super line graph transformation.

7 Considerations

Given Remark for k -regular graphs I could obtain a computationally faster result by computing the WL on $L_2(G)$ compared with 3- WL if the graph does not have too much degree variance or mean degree. This is because the 3- WL test needs $\Theta(n^4)$ computational time per step, whereas the $WL(L_2(G))$ takes $O(|V_{L_2}| + |E_{L_2}|) = O(n^3)$ for k -regular graphs.

For real-world datasets, I gathered interesting results. In fact, plotting the value of $|E_{L_2}|$ against n^4 ($n = |V|$) (Figure 1) for famous real-world dataset benchmarks (MUTAG, NCI1, ZINC, ...) revealed that n^4 was clearly larger than the actual $|E_{L_2}|$. This provided hope for using architectures built on the super line graph transformation instead of models relying on 3- WL , which costs $\Theta(n^4)$.

We can notice that the super line graph creates a set of three or four nodes for each node in the transformation and it intuitively makes sense that the test $WL(L_2())$ is a mix of 3- WL and 4- WL , even though I have not been able to show that it is at most as expressive as 4- WL .

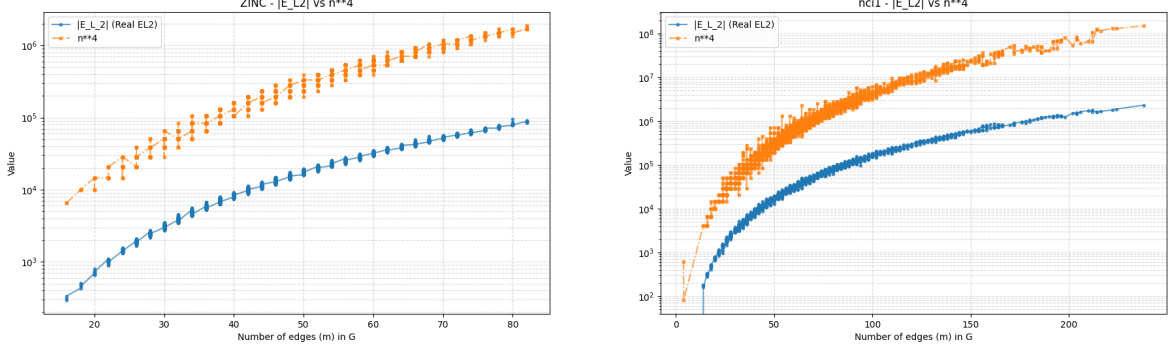


Figure 1

The super line graph transformation tends to produce complete graphs for very small networks. In fact (Theorem 15.15 [2]), if G does not contain $2P_3$ or $3K_2$ as subgraphs, then $L_2(G)$ is complete. Thanks to this theorem, we can find infinite cases, for example: $K_{1,3}$ and K_3 could be distinguished by 3-WL but not by $WL(L_2(G))$.

8 Furure work

I would like to find some theoretically grounded results regarding the expressivity of $WL(L_2(\cdot))$, find an upper bound to $|E(L_2(\cdot))|$ that adapts for all the cases. Continuing experimentig on the creation of a model based on this transformation, and test it in some benchmark of subgraphs counting.

A Appendix

A.1 Proof of Lemma 2.1

We consider the same notation as in the Lemma. The idea is to find a bound to the degree of a node in V_{L_2} and then use the handshake lemma to find a bound to $|E_{L_2}|$. Given a node $A = \{e_i, e_j\} \in V_{L_2}$, we define $I(A)$ as the number of edges in E incident to e_i or e_j . To use a simple notation, we consider $(e_i, e_j) = (\{u_1, u_2\}, \{u_3, u_4\})$. $I(A) = \sum_{i=1}^4 d(u_i) - x$ where x is a positive integer because there can be repetitions in the count of incident edges for one of the four nodes so $I(A) \leq \sum_{i=1}^4 d(u_i)$. The degree of node A is obtained by considering all the nodes that are different from A and subtracting all the non incident ones.

$d(A) = \text{total pairs of edges} \setminus \{A\}$ - pairs where both of the edges are not in $I(A)$.

$$d(A) = \binom{m}{2} - 1 - \binom{m - I(A)}{2} = \dots = mI(A) - x \leq mI(A)$$

where $(x > 0)$. Thus, $d(A) \leq mI(A)$, and

$$|E_{L_2}| = \frac{1}{2} \sum_{A \in V_{L_2}} d(A) \leq \frac{1}{2} \sum_{A \in V_{L_2}} I(A)m \leq \frac{m}{2} \sum_{A \in V_{L_2}} \left(\sum_{v \in e \in A} d(v) \right).$$

Now $\sum_{v \in e \in A} d(v) = \sum_{\{u_1, u_2\} \in E, \{v_1, v_2\} \in E} d(v_1) + d(v_2) + d(u_1) + d(u_2) \stackrel{(1)}{=} (m-1) \sum_{\{u_1, u_2\} \in E} d(u_1) + d(u_2) = (m-1) \sum_{v \in V} d(v)^2$. The equality (1) follows from counting every edge $m-1$ times. The last equality is justified by Theorem 1.1 of the line graph book [2]. Now $\sum_{v \in V} d(v)^2 = \frac{2m}{M}(\sigma^2 + M^2)$. By putting everything together, we obtain $|E_{L_2}| \leq O(m^3 M + \frac{m^3 \sigma^2}{M})$. Remember that $m = Mn/2$. $\square$.

A.2 Proof of Lemma 2.3

Let $G = G(n, p)$ be an Erdős-Rényi random, where $V = \{v_1, \dots, v_n\}$ are the nodes. Every edge has an independent probability of p of existing, so every edge $\{v_i, v_j\}$ depends on a r.v. $X_{ij} \sim \text{Ber}(p)$ for $1 \leq i < j \leq n$, where $X_{ij} = 1$ if $\{v_i, v_j\} \in E$ and 0 otherwise. For convenience, we define $X_{ji} := X_{ij}$.

The total number of edges is $m = \sum_{i < j} X_{ij} \sim \text{Bin}\left(\frac{n(n-1)}{2}, p\right)$. The mean degree is $M = \frac{2m}{n}$, and the degree of a vertex v_i is $d(v_i) = \sum_{j \neq i} X_{ij} \sim \text{Bin}(n-1, p)$. The sample variance of the degrees is $\sigma^2 = \frac{1}{n} \sum_{i=1}^n d(v_i)^2 - M^2$.

We can use the previous bound to get the expected number of edges of $L_2(G(n, p))$

$$\mathbb{E}[|E_{L_2}(G)|] = \mathbb{E}[O(n^3 M^4 + n^3 M^2 \sigma^2)] = O(n^3 \mathbb{E}[M^4] + n^3 \mathbb{E}[M^2 \sigma^2]). \quad (1)$$

First, considering the fourth moment of the binomial distribution m , we have:

$$\mathbb{E}[M^4] = \frac{16}{n^4} \mathbb{E}[m^4] = O(n^{-2}p + p^2 + n^2 p^3 + n^4 p^4),$$

Then the first term of the Equation (1) is:

$$n^3 \mathbb{E}[M^4] = O(np + n^3 p^2 + n^5 p^3 + n^7 p^4).$$

For the second term, we can rewrite the sample variance as $\sigma^2 = \frac{1}{2n^2} \sum_{i=1}^n \sum_{j \neq i} (d(v_i) - d(v_j))^2$. Then we get

$$\begin{aligned} \mathbb{E}[\sigma^2 M^2] &= \frac{n(n-1)}{2n^2} \mathbb{E}[(d(v_1) - d(v_2))^2 M^2] = \\ &= \frac{n(n-1) \cdot 4}{2n^4} \mathbb{E}[(d(v_1) - d(v_2))^2 m^2] \leq \frac{2}{n^2} \mathbb{E}[(d(v_1) - d(v_2))^2 m^2]. \end{aligned}$$

We can express the expectation as a polynomial of mutually independent random variables. Let $U = \sum_{j \geq 3} X_{1j}$ and $V = \sum_{j \geq 3} X_{2j}$. Both U and V are independently distributed as $\text{Bin}(n-2, p)$. Let $A = X_{12} + \sum_{3 \leq i < j \leq n} X_{ij} \sim \text{Bin}\left(1 + \frac{(n-2)(n-3)}{2}, p\right)$. Notice that A, U , and V are mutually independent. We can write $d(v_1) - d(v_2) = (X_{12} + U) - (X_{12} + V) = U - V$ and $m = A + U + V$, so:

$$\mathbb{E}[(d(v_1) - d(v_2))^2 m^2] = \mathbb{E}[(U - V)^2 (A + U + V)^2].$$

Then we can expand this polynomial $U^2 A^2, U^3 A$, etc. Since A, U , and V are independent (e.g., $\mathbb{E}[U^2 A] = \mathbb{E}[U^2] \mathbb{E}[A]$). We can compute the moments of those binomial distributions and get only the dominant terms

$$\mathbb{E}[(U - V)^2 (A + U + V)^2] = O(np + n^3 p^2 + n^5 p^3).$$

We obtain the bound for the second term:

$$n^3 \mathbb{E}[M^2 \sigma^2] = O(n^2 p + n^4 p^2 + n^6 p^3).$$

Then we can conclude:

$$\mathbb{E}[|E_{L_2}(G)|] = O(n^7 p^4 + n^6 p^3 + n^4 p^2 + n^2 p). \quad \square$$

A.3 Proof of Observation 2.5

We start by defining the base graph. $G_0 = (V_0, E_0)$, $n_0 = |V_0|$, $m_0 = |E_0|$, $1 \leq \bar{m} \leq m_0$, then we can notice that $|V_{t+1}| = |V_t| + 1$, $|E_{t+1}| = |E_t| + \bar{m} = \bar{m}(t+1) + m_0$. As stated in the paper [1] for many steps the system reaches a stationary configuration regarding the distribution of the degree: $\mathbb{P}(d(v) = k) = k^{-\gamma}$ ($\gamma = 3$). Then for $t \gg m_0$ for simplicity we can consider $|V_t| \approx t$, $|E_t| \approx t\bar{m}$. From now on $|V_t| = N_t$, then the mean degree at time t is $M_t = \frac{2(t\bar{m})}{t+n_0} = 2\bar{m}$ and the variance is $\sigma_t^2 = \frac{1}{N_t} \sum d(v)^2 - M_t^2$. Then $\mathbb{E}[\sigma^2] = \sum \mathbb{E}[d(v)^2] - M_t^2$, we can notice that $\sum \mathbb{E}[d(v)^2] = \mathbb{E}[d(v)^2] = \sum_{k=\bar{m}}^{K_{max}} k^2 \mathbb{P}(k) = 2\bar{m}^2 \sum_{k=\bar{m}}^{K_{max}} \frac{1}{k} \approx 2\bar{m}^2 \log\left(\frac{K_{max}}{\bar{m}}\right)$. The last equality is given by this approximation: $2\bar{m}^2 \sum_{k=\bar{m}}^{K_{max}} \frac{1}{k} \leq 2\bar{m}^2 \int_{\bar{m}}^{K_{max}} \frac{1}{k} dk = 2\bar{m}^2 \log\left(\frac{K_{max}}{\bar{m}}\right)$. Now we try to estimate K_{max} . Firstly we get the exact formula of the distribution of the degree. Given $v \in V_t$, $\mathbb{P}(d(v) = k) = Ck^{-3}$ then $\int_{\bar{m}}^{\infty} Ck^{-3} dk = 1 \Rightarrow C = 2\bar{m}^2$. We can now derive a probability bound similar to $\mathbb{P}(\max(v_1 \dots v_N) < p) > 1 - \delta$.

$\mathbb{P}(v_i = k) = k^{-3}2\bar{m}^2$ then $\mathbb{P}(d(v_i) \geq p) = \sum_{k=p}^{\infty} 2\bar{m}^2 k^{-3} \approx 2\bar{m}^2 \int_p^{\infty} x^{-3} dx = \frac{\bar{m}^2}{p^2}$. Now we can use the union bound $\mathbb{P}(K_{max} \geq p) = \mathbb{P}(\bigcup_{i=1}^{N_t} \{d_i \geq p\}) \leq \sum_{i=1}^{N_t} \mathbb{P}(d_i \geq p) \leq N_t \frac{\bar{m}^2}{p^2} \Rightarrow \mathbb{P}(K_{max} < p) \geq 1 - \frac{N_t \bar{m}^2}{p^2} \Rightarrow$ if $p = c \cdot \bar{m} \sqrt{N_t}$ and we get:

$$\mathbb{P}(K_{max} < c\bar{m}\sqrt{N_t}) \geq 1 - \frac{1}{c^2}$$

For convenience, we set $K_{max} = N_t$. Substituting this value back into the expectation:

$$\mathbb{E}[d(v)^2] \approx 2\bar{m}^2 \log\left(\frac{N_t}{\bar{m}}\right) = 2\bar{m}^2 \log(N_t) - 2\bar{m}^2 \log(\bar{m}).$$

Since we are only interested in the order of magnitude, we can ignore the constant factor and approximate the variance $\mathbb{E}[\sigma_t^2]$ as:

$$\mathbb{E}[\sigma_t^2] \approx \bar{m}^2 \log(N_t) - 4\bar{m}^2.$$

We can use $\mathbb{E}[|E_{L_2}(G_t)|] = O(N_t^3 \mathbb{E}[M_t^4] + N_t^3 \mathbb{E}[\sigma_t^2 M_t^2])$. Since M_t is tightly concentrated around $2\bar{m}$, we approximate $M_t^2 \approx 4\bar{m}^2$ and $M_t^4 \approx 16\bar{m}^4$. Substituting $\mathbb{E}[\sigma_t^2] \approx \bar{m}^2 \log(N_t)$:

$$O(N_t^3 \mathbb{E}[M_t^4] + N_t^3 \mathbb{E}[\sigma_t^2 M_t^2]) = O(N_t^3 \bar{m}^4 + N_t^3 (4\bar{m}^2)(\bar{m}^2 \log(N_t))) = O(N_t^3 \bar{m}^4 + 4N_t^3 \bar{m}^4 \log(N_t)).$$

Then for $N_t \rightarrow \infty$, the expression simplifies to:

$$\mathbb{E}[|E_{L_2}(G_t)|] = O(\bar{m}^4 N_t^3 \log(N_t)). \quad \square$$

A.4 Proof sketch Lemma 2.6

The idea is that we know that the degree of a node in L_2 depends only on the set $I(A)$ ($|E_{L_2}(G)| = f(I(A), m)$) defined in the proof of Lemma 2.1. In such a regular graph, we can count all possible configurations of $I(A)$ that depend on the positions of the two edges e_1, e_2 in A , of which there are only four. If they are incident, if they are too far away, or if they are at a distance of one (only an edge between e_1 and e_2), and in the last case they can be in opposite positions in a cycle C_4 or not. In the first case $I(A) = 3k - 4$, if they are far away $I(A) = 4k - 4$. If they are at a distance of one but not in an induced subgraph of C_4 , then $I(A) = 4k - 5$. In the case of an opposite position in an induced subgraph C_4 then $I(A) = 4k - 6$. To find this number we have just to count the number of distinct nodes in A , count the degree of each one and subtract the repeated edges in the count. The number of nodes in L_2 that have that degree ($4k - 6$) will be different in G_1 and G_2 if they have a different number of induced C_4 . We can notice that even if k is such that $4k - 6 = 3k - 4$, with $k = 2$, the number of nodes with the degree $f(I(A))$ will be different. Maybe the same reasoning can be applied to other k -regular graph with a specific different substructure, and by isolating all the possible configuration of two edges and by counting $I(A)$ we can find a different number of nodes with a certain degree.

A.5 Algorithm complexity

Notice that the naive solution to compute $L_2(G)$ is to first compute the list of the nodes and then check for each pair of nodes A, B if an edge in A is incident with an edge in B . This solution has a runtime, for a total of $\binom{m}{2} = \Theta(m^2)$ computational time, that in the worst case (complete graph K_n) is $\Theta(n^2)$.

We want a method that is able to compute $L_2(G)$ in $O(|V_{L_2}| + |E_{L_2}|)$ time, which is the best we can do since it is the size of the output.

A.5.1 Algorithm for E_{L_2}

We compute all the nodes of $L_2(G)$ and we assign an ID to each node (starting from zero). We get the node A with the minimum ID in V_{L_2} and we create a set with all the edges incident to the edges in A , that we call $N(A)$. Given this set we can build all the edges where A is present by getting all the pairs of edges that have at least one edge in $N(A)$ and compose those pairs with A to have the edge of $L_2(G)$. In the next step we get a node B with the next ID, and we check $N(B)$ and we use the same process, but we add the edges in $E_{L_2}(C, B)$ only if $ID(C) > ID(B)$, avoiding the creation of redundant edges. The complexity matches the number of nodes and the number of edges of $L_2(G)$, such that $O(|V_{L_2}| + |E_{L_2}|)$.

This function has been implemented in C++ and integrated into Python with pybind11. It has been tested and validated with brute-force solutions.

B Tests

To validate the theoretical bound on $|E_{L_2}|$ for the BA model we don't want to use the computational cost of explicitly constructing L_2 for very large graphs. Then we empirically verify σ_t^2 for each step. Since M_t is clearly computed, we only care about σ_t^2 .

We consider an initial star graph $K_{1,200}$, which is the worst-case initialization for the degenerative case. From this base graph, we generate the BA model for 50000 steps with different values of $\bar{m}$. When $\bar{m} = 1$, the process simulates a degenerate case, for large $\bar{m}$ the standard BA behavior emerges.

Figure 2 displays five scenarios with $\bar{m} \in \{1, 5, 20, 50, 100\}$. The empirical variance matches the estimated value $\mathbb{E}[\sigma_t^2] < 2\bar{m}^2 \log(N_t) - 4\bar{m}^2$, confirming our theoretical derivations for large $\bar{m}$.

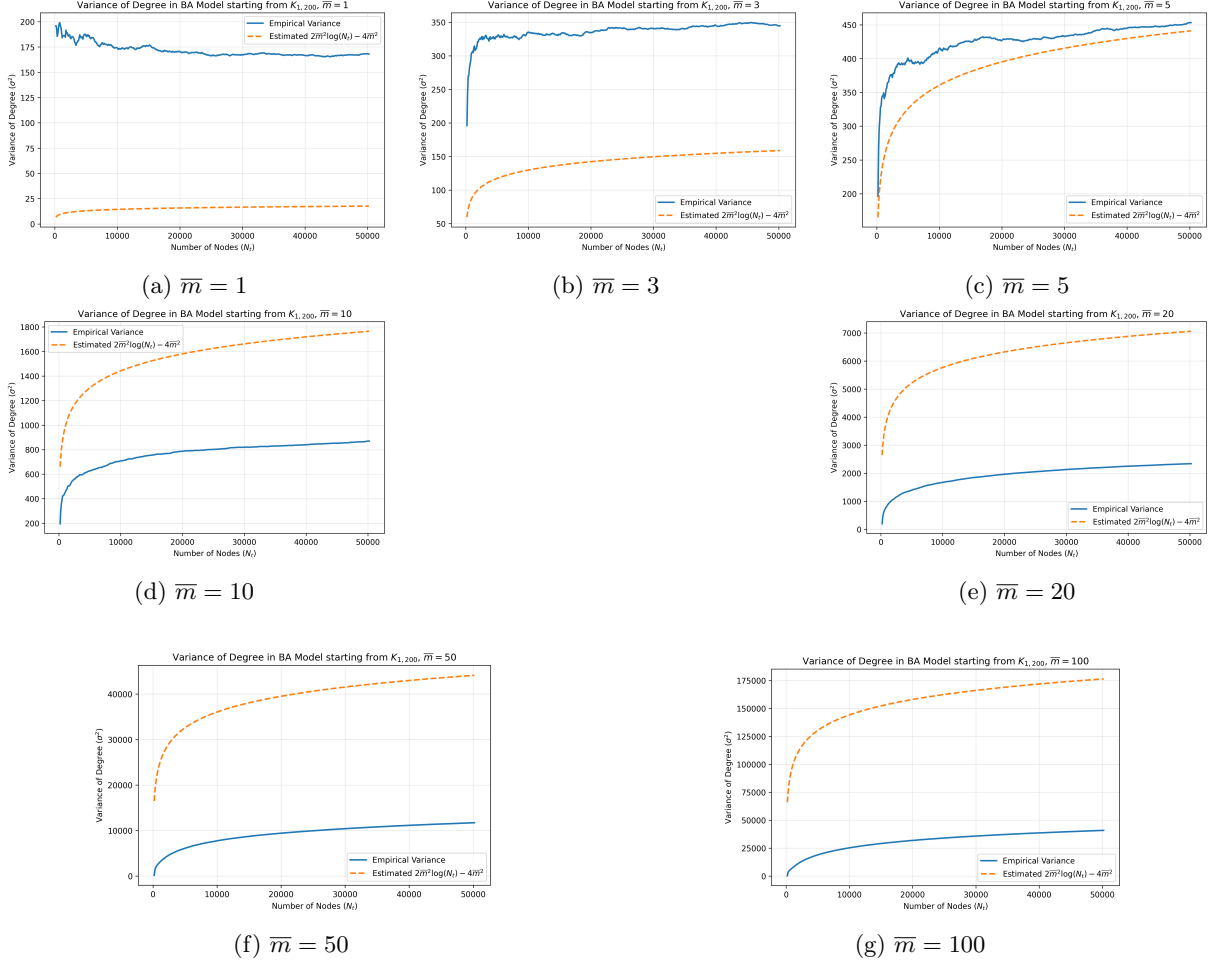


Figure 2: Empirical and estimated degree variance σ^2 of a BA model for 50000 steps, starting from a base graph $K_{1,100}$.

References

- [1] Aleksandr Abramov, Uliana Gorik, Andrei Velichko, Vladimir Nelyub, Aleksandr Samoshkin, Andrei Gantimurov, Aleksei Borodulin, Vadim S. Tynchenko, and Ivan Malashin. Barabási–albert-based network growth model to sustainable urban planning. *Sustainability*, 17(3), 2025.
- [2] Lowell Beineke and Jay Bagga. *Line Graphs and Line Digraphs*. 01 2021.
- [3] Lin Du, Lu Bai, Jincheng Li, Lixin Cui, Hangyuan Du, Lichi Zhang, Yuting Chen, and Zhao Li. Lgan: An efficient high-order graph neural network via the line graph aggregation, 2025.

- [4] Ningyuan Teresa Huang and Soledad Villar. A short tutorial on the weisfeiler-lehman test and its variants. In *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, page 8533–8537. IEEE, 2021.
- [5] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks, 2021.
- [6] Jusuf Qarkaxhija, Engin Melekoglu, and Murat Gök. Lg-gnns: Enhancing node classification via line graph transformations for higher-order structural learning. *Turkish Journal of Engineering*, 9:670–677, 09 2025.
- [7] Manizheh Ranjbar, Mahdi Jalili, Xiaodong Li, and Parham Moradi DW. MLGLP: Multi-scale line-graph link prediction based on graph neural networks, 2025.
- [8] Yanbo Wang and Muhan Zhang. An empirical study of realized gnn expressiveness, 2024.
- [9] Fan Yang and Xingyue Huang. Theoretical insights into line graph transformation on graph learning, 2025.
- [10] Zehua Zhang, Shilin Sun, Guixiang Ma, and Caiming Zhong. Line graph contrastive learning for link prediction, 2023.